

난독화된 한글 리뷰 복원 AI 프로젝트 보고서

C팀 김소연, 김유진, 이유림

1. 프로젝트 개요

본 프로젝트의 목표는 난독화된 한글 리뷰를 원문 리뷰로 복원하는 AI 모델을 구현하는 것이다. 최근 온라인 커뮤니티와 SNS에서는 초성, 은어, 발음 변형, 오타 등을 활용한 난독화 표현이 자주 사용된다. 이러한 문장은 사람은 이해할 수 있지만 컴퓨터는 해석하기 어렵기 때문에 자연어처리(NLP) 관점에서 흥미로운 문제로 볼 수 있다.

본 프로젝트에서는 Rule-Based 방식인 Dictionary Baseline과 생성형 AI 기반의 LLM Baseline을 비교하고, 각각의 성능 개선 실험을 수행하였다.

2. 데이터 이해

2.1 데이터 구성

제공된 데이터는 다음과 같이 구성되어 있다.

- train.csv
- test.csv
- sample_submission.csv

Train 데이터는 난독화 문장(input)과 정답 문장(output)을 모두 포함하고 있으며, Test 데이터는 난독화 문장만 제공된다.

2.2 난독화 리뷰의 특징

데이터를 분석한 결과 다음과 같은 특징을 확인할 수 있었다.

- 발음 기반 변형이 많음
- 거센소리 및 된소리 변형 존재
- 실제 한국어에 없는 받침 사용

- 의미는 유지되지만 표기가 변형됨
- 초성 및 인터넷 은어 빈번하게 등장
- 문장 구조 자체는 유지되는 경우가 많음

예시

원문	난독화
최고	ㄸ ㄱ
좋다	조타
가성비	갓성비
정말	ㅈ ㅁ

2.3 평균 문장 길이

문장은 짧은 문장부터 긴 문장까지 다양하게 분포하였다. 평균적으로 약 2문장 정도로 구성되어 있었으며, 첫 문장은 짧은 감상, 두 번째 문장은 구체적인 후기인 경우가 많았다

2.4 자주 등장하는 난독화 유형

유형	예시
거센소리화	ㄱ → ㅋ
된소리화	ㄷ → ㄸ
받침 추가	가 → 각
모음 변형	ㅏ → ㅑ
초성 축약	최고 → ㄸ ㄱ
인터넷 은어	존맛, 개꿀, 갓성비

2.5 문자 단위 평가 방식

본 프로젝트에서는 Character F1 Score를 사용하였다. Character F1은 복원된 문장과 정답 문장을 문자 단위로 비교하여 Precision과 Recall을 계산한 뒤 F1 Score를 산출하는 방식이다. 문장 전체가 완벽히 일치하지 않더라도 일부 문자를 올바르게 복원한 경우 점수

를 받을 수 있다는 장점이 있다.

3. 전체 실행 흐름 설명

프로젝트 전체 실행 흐름은 다음과 같다.

데이터 불러오기

→ 텍스트 전처리

→ Dictionary 구축

→ Tokenizer 구성

→ Dataset 구성

→ 모델 정의

→ 학습

→ 검증

→ 테스트 추론

→ 제출 파일 생성

각 단계는 데이터 품질 향상, 모델 학습 효율 증가, 최종 성능 향상을 위해 필요하다.

4. 데이터 전처리 및 증강

4.1 데이터 전처리

난독화 데이터의 특성을 고려하여 다음과 같은 전처리를 수행하였다.

- 특수문자 제거
- 연속 공백 제거
- 초성 및 중성 보존
- 반복 문자 정규화
- 입력과 출력 데이터에 동일한 정제 기준 적용

이를 통해 의미 정보는 유지하면서 불필요한 노이즈를 줄일 수 있었다.

4.2 데이터 증강

모델의 일반화 성능 향상을 위해 난독화 패턴을 추가 생성하였다.

적용한 증강 방법은 다음과 같다.

- 띄어쓰기 변형
- 초성 기반 변형
- 발음 변형
- 오타 삽입
- 특수문자 추가
- 반복 문자 추가

예시

원문	증강 결과
좋아요	좋 아 요
최고	ㄸ ㄱ
좋다	조타
좋아요	좋아용ㅋㅋ

증강 후 데이터 수는 기존 데이터의 약 2배 수준으로 증가하였다.

5. Dictionary Baseline 분석 및 개선

5.1 Baseline 구조

Dictionary Baseline은 Train 데이터의 input-output 관계를 이용하여 단어 사전을 구축하고, 입력 문장을 사전 기반으로 치환하는 방식이다.

예시

- ㅈㅊ → 최고
- ㅊㅈ → 감사

5.2 Exact Matching 개념

기존 Baseline은 Exact Matching을 사용했다. Dictionary에 저장된 단어와 입력 단어가 완전히 동일해야 복원이 가능하다.

예를 들어

- ㅈㅊ → 최고

는 복원 가능하지만

- ㅈㅊ임
- ㅈㅊ이다

와 같은 표현은 복원하지 못한다.

5.3 성능 개선 실험

- 실험 1: 전처리 개선

safe_preprocess 함수를 적용하였다.

적용 내용

- 특수문자 제거
- 초성 및 중성 보존
- 연속 공백 제거
- 반복 문자 정규화

- 실험 2: Dictionary 확장

초성 및 인터넷 은어를 수동 Dictionary에 추가하였다.

예시

- ㅈㅊ → 최고
- ㅊㅈ → 감사
- 존맛 → 정말 맛있다
- 강추 → 추천한다

- 실험 3: 부분 문자열 매칭

기존 Exact Matching 방식의 한계를 보완하기 위해 부분 문자열 매칭을 적용하였다.

예시

- 존맛탱구리
- 개추천임

과 같은 표현도 일부 복원이 가능하도록 개선하였다.

5.4 실험 결과

실험	Validation Char F1
Baseline Dictionary	0.2366
전처리 적용	0.2799
Dictionary 확장 + 부분 문자열 매칭	0.3179

Dictionary 개선을 통해 약 34%의 성능 향상을 확인하였다.

5.5 Rule-Based 방식의 장점

- 구현이 단순함
- 학습 시간이 필요 없음
- 자주 등장하는 패턴에 강함
- 해석이 쉬움

5.6 Rule-Based 방식의 한계

- 새로운 표현 복원이 어려움
- 문맥을 이해하지 못함
- 학습 데이터에 없는 표현 처리 불가
- 일반화 성능이 낮음
- 학습 데이터에 존재하지 않는 새로운 난독화 표현은 복원이 거의 불가능함

6. LLM Baseline 설명

6.1 LLM Baseline

LLM Baseline은 HuggingFace에서 제공하는 KoBART(gogamza/kobart-base-v2) 모델을 활용하였다. KoBART는 한국어 텍스트 생성에 특화된 Encoder-Decoder 구조의 사전학습 언어모델로, 입력 문장의 문맥을 이해하여 자연스러운 문장을 생성할 수 있다.

6.2 Tokenizer

Tokenizer는 입력 문장을 토큰 단위로 분리하고 모델이 처리할 수 있는 숫자 형태로 변환한다.

6.3 Encoder-Decoder 구조

Encoder는 입력 문장의 의미를 벡터로 변환한다. Decoder는 Encoder의 정보를 이용하여 복원 문장을 생성한다.

6.4 Pretrained 모델

사전학습 모델은 대규모 한국어 데이터로 미리 학습되어 있어 풍부한 언어 지식을 보유하고 있다. 이를 활용하면 적은 데이터로도 높은 성능을 얻을 수 있다.

6.5 Fine-Tuning

성능 향상을 위해 하이퍼파라미터 튜닝을 수행하였다.

변경한 주요 항목은 다음과 같다.

- Learning Rate : $1e-5$
- Batch Size : 16
- Epoch : 5

이를 통해 학습 안정성과 일반화 성능을 향상시킬 수 있었다.

6.6 Validation 결과

Epoch별 Validation Char F1 결과는 다음과 같다.

Epoch	Validation Char F1
1	0.5001
2	0.5216
3	0.5357
4	0.5419
5	0.5444

최종 Validation Char F1은 0.5444를 기록하였다.

7. 실험 결과 비교

실험 번호	모델	Pretrained	Augmentation	F1
Exp1	Dictionary Baseline	X	X	0.2366
Exp2	Dictionary + 전처리	X	X	0.2799
Exp3	Dictionary + 사전확장 + 부분매칭	X	X	0.3179
Exp4	KoBART Baseline	O	X	약 0.14

Exp5	KoBART + 증강 + 튜닝	O	O	0.5444
------	------------------	---	---	--------

8. 결과 해석

Dictionary 방식은 구현이 간단하고 빠르지만 새로운 표현에 대한 일반화 능력이 부족하였다. 반면 LLM 방식은 문맥 정보를 활용할 수 있기 때문에 난독화된 문장을 더욱 자연스럽게 복원할 수 있었다. 데이터 증강은 다양한 난독화 패턴을 학습하게 만들어 모델의 일반화 성능 향상에 기여하였다. 특히 증강 데이터를 활용한 KoBART 모델은 Validation Char F1 0.5444를 기록하여 Baseline 대비 큰 폭의 성능 향상을 보였다. 가장 어려웠던 난독화 패턴은 초성, 발음 변형, 은어가 동시에 적용된 경우였다. 추가적으로 BLEU Score와 Edit Distance를 활용한 오류 분석을 수행하였다. 분석 결과 초성 축약, 발음 변형, 인터넷 은어가 동시에 포함된 문장에서 오차가 크게 발생하는 경향을 확인하였다. 또한 문장 길이가 길어질수록 Edit Distance가 증가하는 경향을 보였다.

9. 팀원별 기여도

이름	담당 역할	실제 수행 내용	이해한 핵심 개념
김소연	데이터 분석 및 전처리	데이터 분석, 전처리 설계, 데이터 증강	데이터 전처리, Augmentation
이유림	Dictionary Baseline 분석 및 개선	Dictionary 구축, 전처리 적용, 사전 확장, 부분 문자열 매칭 구현, 성능 평가	Rule-Based 방식, Dictionary 구축, Character F1
김유진	LLM Baseline 및 성능 개선	모델 학습, Fine-Tuning, 하이퍼파라미터 조정	Tokenizer, Encoder-Decoder, LLM

10. 생성형 AI 사용 기록

항목	기록
어떤 질문을 했는가?	Dictionary Baseline의 성능을 향상시키기 위한 방법, Rule-Based 방식의 한계를 보완할 수 있는 방법, Character F1 Score 계산 방식, 부분 문자열 매칭 구현 방법, 전처리 적용 시 성능 변화 분석, LLM 모델 학습 과정에서 정규화를 어떻게 적용해야 하는지와 과적합을 방지하기 위한 방법에 대해 질문하였다.
어떤 코드 생성 또는 수정에 도움을 받았는가?	특수문자 제거 및 반복 문자 정규화 전처리 코드, Dictionary 확장 코드, 초성-은어·축약어 사전 추가 코드, 부분 문자열 매칭 로직, Validation Character F1 계산 코드 작성에 도움을 받았다.
AI가 생성한 코드 중 직접 수정한 부분은 무엇인가?	난독화 데이터에서 자주 등장하는 표현(ㄸ, ㅅ, 존맛, 갓성비, 강추, 노잌 등)을 직접 추가하였으며, 팀 A의 전처리 코드(safe_preprocess)를 적용하여 Dictionary 구축 과정을 수정하였다. 또한 부분 문자열 매칭 범위와 Dictionary 항목을 직접 보완하였다.
이해되지 않았던 코드와 해결 과정은 무엇인가?	Exact Matching 방식이 왜 새로운 난독화 표현을 복원하지 못하는지 이해하기 어려웠으나, 학습 데이터에 존재하지 않는 단어는 Dictionary에 등록되지 않기 때문이라는 점을 확인하였다. 또한 Character F1 Score 계산 방식과 부분 문자열 매칭 로직을 예제를 통해 이해하였다.
최종 코드의 실행 흐름을 설명할 수 있는가?	데이터 로드 → Train/Validation 분리 → 텍스트 전처리 → Dictionary 구축 → 추가 Dictionary 확장 → 문장 단위 및 단어 단위 매칭 → Validation Character F1 평가 → Test 데이터 복원 → submission 파일 생성의 전체 흐름을 설명할 수 있다.